

Lab Report: Parallel Skeletons

Student Name: Sharique Baig

ERP: 28369

Course: Parallel and Distributed Computing

Date: February 19, 2026

1. Introduction

This lab explores **Parallel Skeletons**, which are high-level programming patterns that abstract parallel computations. These skeletons facilitate structured parallel programming by separating the "logic" of the task from the "mechanics" of parallel execution.

We implemented and analyzed two categories:

1. **Data Parallel Skeletons:** Map, Reduce, Scan.
 2. **Task Parallel Skeletons:** Divide and Conquer, Wavefront.
-

2. Experimental Setup

- **Language:** Python 3.12
 - **Hardware:** Multi-core processor (utilizing `multiprocessing` and `concurrent.futures`)
 - **Metric:** Execution time measured using `time.perf_counter()`.
-

3. Lab Tasks & Results

Task 1: Map Skeleton (Element-Wise Squaring)

- **Skeleton:** Data Parallel Map.
- **Objective:** Apply a heavy math operation ($\sqrt{x} + \sin(x)$) to 5,000,000 numbers.
- **Implementation:** Used `multiprocessing.Pool.map`.
- **Results:**
 - **Sequential:** 2.76s
 - **Parallel:** 1.30s
 - **Outcome: 2.1x Speedup.** Parallelism thrived because each calculation was independent.

Task 2: Reduce Skeleton (Parallel Sum)

- **Skeleton:** Data Parallel Reduce.
- **Objective:** Sum 5,000,000 processed numbers.
- **Implementation:** Used "Chunking" where multiple cores compute partial sums, which are then combined.
- **Results:**
 - **Sequential:** 1.99s
 - **Parallel:** 1.07s

- **Outcome: 1.8x Speedup.** Efficiency was gained by dividing the reduction workload across cores.

Task 3: Scan Skeleton (Prefix Sum)

- **Skeleton:** Data Parallel Scan.
- **Objective:** Compute all partial sums of an array.
- **Implementation:** Hillis-Steele Algorithm (Step-wise doubling).
- **Results:**
 - **Sequential:** 0.000069s
 - **Parallel (Simulated):** 0.003853s
- **Outcome:** Demonstrated $O(\log N)$ parallel steps logic. While simulated in Python with some overhead, the algorithm allows for massive scalability on hardware like GPUs.

Task 4: Parallel Merge Sort (Divide & Conquer)

- **Skeleton:** Task Parallel D&C.
- **Objective:** Sort 500,000 random integers.
- **Implementation:** Parallelized the top-level splits of the sorting tree using `ProcessPoolExecutor`.
- **Results:**
 - **Sequential:** 2.48s
 - **Parallel:** 1.51s
 - **Outcome: 1.6x Speedup.** The "Parallel Threshold" ensured we didn't waste time on process overhead for small chunks.

Task 5: Parallel LCS (Wavefront)

- **Skeleton:** Task Parallel Wavefront.
- **Objective:** Find Longest Common Substring with heavy comparison work.
- **Implementation:** Diagonal processing (all cells (i, j) where $i+j=k$ are processed in parallel).
- **Results:**
 - **Sequential:** 85.41s
 - **Parallel:** 25.94s
 - **Outcome: 3.3x Speedup.** This task benefited most from parallelism due to the high computational intensity of each cell calculation.

4. Conclusion

Parallel Skeletons provide a powerful abstraction for complex systems. Through this lab, I observed that:

- **Scalability** depends on task complexity; simple tasks are better left sequential due to IPC overhead.
- **Data Locality** and chunking are essential for skeletons like Reduce and Map.
- **Dependencies** (as seen in Wavefront) don't prevent parallelism but require smart ordering (diagonals).

All codes were verified for correctness and performance benchmarks were successfully met.